Lesson: Functions

CSE 4107 : Structured Programming I

Shahriar Ivan

Introduction

- A ***function*** is a series of statements that have been grouped together and given a name
- Each ***function*** is essentially a small program, with its own declarations and statements
- Advantages of functions:
  - A program can be divided into small pieces that are easier to understand and modify
  - We can avoid duplicating code that's used more than once
  - A ***function*** that was originally part of one program can be reused in other programs

Programming Task

Computing Averages

- The program reads three numbers and uses the average function to compute their averages, one pair at a time:

```
Enter three numbers: 3.5 9.6 10.2
Average of 3.5 and 9.6: 6.55
Average of 9.6 and 10.2: 9.9
Average of 3.5 and 10.2: 6.85
```

Programming Task

Computing Averages

- Return type ⇒ double
  - Parameters ⇒ a, b
  - Body ⇒ Executable, enclosed in braces
    - Contains a return statement
      - » Returns to the place from which it was called
      - » Returns the value (a+b)/2

Programming Task

Computing Averages

- A ***function*** call consists of a function name followed by a list of ***arguments***
  - *average(x, y)* is a call of the average function
- Arguments are used to supply information to a function
  - The call *average(x, y)* causes the values of *x* and *y* to be copied into the parameters *a* and *b*
- An argument doesn't have to be a variable; any expression of a compatible type will do
  - *average(5.1, 8.9)* and *average(x/2, y/3)* are legal

Programming Task

Printing a Countdown

- A function to print the countdown message
- Program will run a for loop and call the function to print the message
- Pass the countdown value as parameter
- void ⇒ Indicates the function has no return value

Programming Task

Hello, World

- A function to print the statement
- Program will call the function to print it
- Doesn’t require any information ⇒ No parameters
- Write only function’s name, followed by parentheses `print_hello();`
- Parentheses must be present

Defining and Calling Functions

Functions Definitions

- General form of a ***function definition:***

```
return-type function-name ( parameters )
{
  declarations
  statements
}
```

Defining and Calling Functions

Functions Definitions

- The *return* type of a ***function*** is the type of value that the ***function*** returns
- Rules governing the *return* type:
  - Functions may not return arrays
  - Specifying that the return type is void indicates that the ***function*** doesn't *return* a value
- If the *return* type is omitted in C89, the ***function*** is presumed to return a value of type int
- In C99, omitting the return type is illegal

Defining and Calling Functions

- function-name ⇒ Identifier
- parameters
  - Each parameter is preceded by specification of its type
  - Separated by commas
  - No parameters ⇒ void
- function-body
  - May include both declaration and statements
  - Variable of one function cannot be accessed by other functions
  - Might be empty if *return* type is void ⇒ During program development

Defining and Calling Functions

- A function call consists of a function name followed by a list of arguments, enclosed in parentheses:

```
average(x, y)
print_count(i)
print_pun()
```

- If the parentheses are missing, the function won't be called:

```
print_pun;              /*** WRONG ***/
```

This statement is legal but has no effect

Defining and Calling Functions

Functions Calls

- void function is always followed by a *semicolon* to turn it into a statement:

```
print_count(i);
print_pun();
```

- A call of a non-void function produces a value that can be stored in a variable, tested, printed, or used in some other way:

```
avg = average(x, y);              // Stored in variable
if (average(x, y) > 0)            // Tested
      printf("Average is positive\n");
printf("The average is %g\n", average(x, y));       // Printed
```

Defining and Calling Functions

Functions Calls

- Ignoring the *return* value of average is an odd thing to do, but for some functions it makes sense
- printf returns the number of characters that it prints
- After the following call, ***num_chars*** will have the value 34:

```
num_chars = printf("Life is like a box of chocolates!\n");
```

- We'll normally discard printf's return value:

```
printf("Life is like a box of chocolates!\n");          /* discards return value */
```

Defining and Calling Functions

Functions Calls

- To make it clear that we're deliberately discarding the return value of a function, C allows us to put (void) before the call:

```
(void) printf("Life is like a box of chocolates!\n");
```

- Using (void) makes it clear to others that you deliberately discarded the return value, not just forgot that there was one

Programming Task

Testing Whether a Number is Prime

- Given a number, tests whether that number is *prime* or not

```
Enter a number: 34

Not prime
```

- Function takes the number as parameter
- Returns true if its prime, false otherwise

Function Declarations

- C doesn’t require that the definition of a function precede its calls
- Suppose that we rearrange the *average.c* program by putting the definition of average *after* the definition of `main`
- When the compiler encounters the first call of *average* in `main`, it has no information about the function
- Instead of producing an error message, the compiler assumes that *average* returns an `int` value
- We say that the compiler has created an ***implicit declaration*** of the function

Function Declarations

- The *compiler* is unable to check that we're passing *average* the right number of arguments and that the arguments have the proper *type*
- Instead, it performs the *default* argument promotions and hopes for the best
- When it encounters the definition of *average* later in the program, the compiler notices that the function's return type is actually double, not int, and so we get an error message

Function Declarations

- One way to avoid the problem of *call-before-definition* is to arrange the program so that the *definition* of each function precedes all its calls
- Unfortunately, such an arrangement doesn't always exist
- Even when it does, it may make the program harder to understand by putting its function definitions in an unnatural order
- Fortunately, C offers a better *solution*: declare each function before calling it

Function Declarations

- Also known as ***function prototypes***
- General Form

    return-type function-name (parameters) ;

- Must be consistent with the function's definition
- Doesn't need to specify the names of parameters

```
double average(double, double);
```

Function Declarations

- Best not to omit parameter names
  - Helps document the purpose of each parameters
  - Reminds the programmer of the order in which arguments must appear in function call
- C99 produces error for implicit declarations

Arguments

- ***Passed by value*** ⇒ Evaluate argument and assign its value to corresponding parameter
- Changes made in parameter doesn't affect the argument
  - Advantage: Can use parameter as variables in function
  - Disadvantage: Might require editing arguments

    » Returning multiple values

```
int power(int x, int n){
        int result = 1;
        while (n-- > 0)
                result *= x;
        return result;
}
```

Arguments

Argument Conversions

- Type of argument might not match the type of the parameters
- Depends on whether compiler has seen
  - Prototype for the function, or
  - Full definition of function prior to the call
- If seen before ⇒ Match with corresponding parameter

  e.g.: int to double for average.c
- If not ⇒ Default argument promotion ⇒ Integer Promotions (C99)/Integral Promotion (C89), e.g.: float ⇒ double, char/short ⇒ int

Arguments

Argument Conversions

- Relying on the default argument promotion is ***dangerous*** ⇒ Undefined behavior

```
#include <stdio.h>
int main(){
        printf("%f", square(5.0));
}
int square(int x){
        return x * x;
}
```

- Solution: Provide prototype before calling it

Arguments

Array Arguments

```
int f(int a[]){                    // no length specification required
        ...
}
```

- Difficult to determine the length ⇒ Supply as an additional argument

```
int sum_array(int a[], int n){
        int i, sum = 0;
        for (i = 0; i < n; i++)
                sum += a[i];
        return sum;
}
```

Arguments

Array Arguments

- Prototype ⇒ `int sum_array(int a[], int n);`
- Don’t put brackets after array name when passing to function

```
#define LEN 100
int main(void){
        int b[LEN], total;
        ...
        total = sum_array(b, LEN);          // Correct way
        int x = sum_array(b[], LEN);        /*** WRONG ***/
        ...
}
```

Arguments

Array Arguments

- Function can’t check whether we passed the correct array length
- Can be used

```
// sum of first 50 numbers
total = sum_array(b, 50);
```

- Don’t pass larger value ⇒ Undefined behavior

Arguments

Array Arguments

- Function can change array argument

```
void store_zeros(int a[], int n){
        int i;
        for (i = 0; i < n; i++)
                a[i] = 0;
}
```

- A call of

  store_zeros(b, 100);

  can modify the contents of the original array

Arguments
Array Arguments

- Multidimensional arrays ⇒ Only the length of the first dimension may be omitted

```
#define LEN 10
int sum_2D_array(int a[][LEN], int n){
        int i, j, sum = 0;
        for (i = 0; i < n; i++)
                for (j = 0; j < LEN; j++)
                        sum += a[i][j];
        return sum;
}
```

- Can be a nuisance ⇒ Array of pointers

The return Statement

- A *non-void* function must use the return statement to specify what value it will return
- The return statement has the form

```
return expression ;
```

- The expression is often just a constant or variable:

```
return 0;
return status;
```

- More complex expressions are possible:

```
return n >= 0 ? n : 0;
```

The return Statement

- If the type of the expression in a return statement doesn’t match the function’s return type, the expression will be implicitly converted to the return type
  - If a function returns an int, but the return statement contains a double expression, the value of the expression is converted to int

The return Statement

- return statements may appear in functions whose return type is void, provided that no expression is given:

```
return;          /* return in a void function */
```

- Example:

```
void print_int(int i){
        if (i < 0)
                return;
        printf("%d", i);
}
```

Program Termination

return 0;

- Return type of main ⇒ int
  - Status code ⇒ Tested when the program terminates
    - » 0 ⇒ Normal termination
    - » Other than 0 ⇒ Abnormal termination
- Return type of main can be omitted in C89 ⇒ Not allowed inC99
- Good practice to return a status code
- Can terminate program only in the main function

Program Termination

return 0;

- Called to terminate the program

```
exit(0);    //Member of <stdlib.h>
```

- Argument passed has the same meaning as main's return value
- Macros can be used as argument

```
exit(EXIT_SUCCESS);      // successful termination
exit(EXIT_FAILURE);      // abnormal termination
```

  - Value of macros ⇒ Implementation-defined; typical values are 0 and 1, respectively

- Can terminate program whenever called

Recursion

- A function is ***recursive*** if it calls itself
- We know:

$$n! = n \times (n - 1)!$$
$$1! = 1$$
$$0! = 1$$

- Use it to define a function:

```
int fact(int n){
        if (n <= 1)       return 1;
        else  return n * fact(n - 1);
}
```

Recursion

- Trace of execution of statement

i = fact(3);

fact(3): 3 ≤ 1 ⇒ Condition false, calls

fact(2): 2 ≤ 1 ⇒ Condition false, calls

fact(1): 1 ≤ 1 ⇒ Condition true, returns 1, causing

fact(2) to return 2 × 1 = 2, causing

fact(3) to return 3 × 2 = 6

Recursion

- The following recursive function computes x^n, using the formula

  $x^n = x \times x^{n-1}$

```
int power(int x, int n){
  if (n == 0)
     return 1;
  else
     return x * power(x, n - 1);
}
```

Recursion

- Condense the power function

```
int power(int x, int n){
        return n == 0  ? 1 : x * power(x, n - 1);
}
```

- Careful to test “*termination condition*” ⇒ Prevents *infinite recursion*